



WEEK 1 - LABORATORY ACTIVITY SHEET s MANUAL-

1. PART 1 – OPERATION COUNTING (WARM-UP)

Pseudocode:

```
Algorithm SampleA(n):  
    total = 0           // operation 1  
    for i = 1 to n:     // n iterations  
        total = total + i // operation 2 per iteration  
    return total        // operation 1
```

Task 1: Operation Counting Table

Fill in the table below based on the pseudocode.

Step / Line	Operation Description	Count
Line 1	Initialization	1
Line 2	Loop setup	1
Line 2	Loop iterations	n
Line 3	Addition operation	n
Line 4	Return	1
TOTAL OPERATIONS		2n+3

Task 2: Convert the total operations into a function

$T(n) = 2n + 3$

Task 3: Simplify to Big-O notation:

$O(n)$





PART 2 – CODE EXECUTION s TIMING SIMULATION

Algorithm A – Linear Search

Java

```
Main.java x untitled1.iml
1 public class Main {
2     @ public static int linearSearch(int[] A, int key) { 1 usage
3         for (int i = 0; i < A.length; i++) {
4             if (A[i] == key) return i;
5         }
6         return -1;
7     }
8
9     public static void main(String[] args) {
10        int[] sizes = {1000, 5000, 10000, 20000};
11
12        for (int n : sizes) {
13            int[] arr = new int[n];
14            for (int i = 0; i < n; i++) {
15                arr[i] = i;
16            }
17
18            long startTime = System.nanoTime();
19            int result = linearSearch(arr, key: n - 1);
20            long endTime = System.nanoTime();
21
22            long duration = endTime - startTime;
23            if (result < 0) System.out.print("");
24            System.out.println("n=" + n + ": " + duration + " ns");
25        }
26    }
27 }
28
```



Algorithm B – Nested Loops

```
Main.java x  untitled1.iml
1  public class Main {
2      public static int nestedLoops(int n) { 1 usage
3          int count = 0;
4          for (int i = 0; i < n; i++) {
5              for (int j = 0; j < n; j++) {
6                  count++;
7              }
8          }
9          return count;
10     }
11
12     public static void main(String[] args) {
13         int[] sizes = {1000, 5000, 10000, 20000};
14
15         for (int n : sizes) {
16             long startTime = System.nanoTime();
17             nestedLoops(n);
18             long endTime = System.nanoTime();
19
20             long duration = endTime - startTime;
21             System.out.println("n=" + n + ": " + duration + " ns");
22         }
23     }
24 }
25
```

Task 4 – Runtime Measurement Table

Steps:

1. Use input sizes: **1,000 | 5,000 | 10,000 | 20,000** (or nearest possible).
2. Measure the execution time using `System.nanoTime()` or similar.
3. Record results in your worksheet.

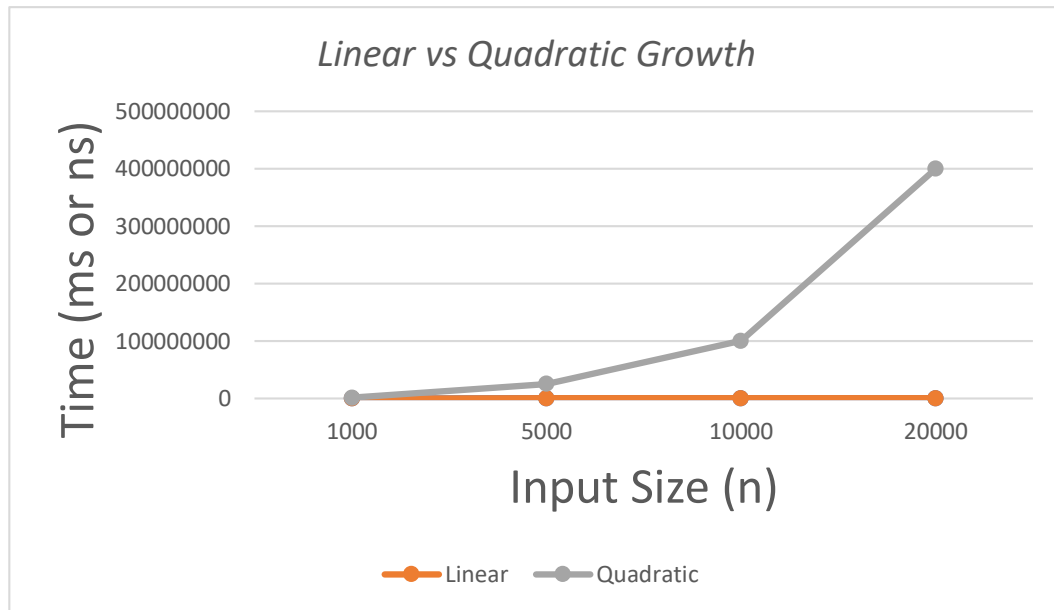
n	Linear Search Time (ns or ms)	Nested Loops Time (ns or ms)
1,000	6900	1000000
5,000	28000	25000000
10,000	55200	100000000
20,000	110700	400000000



PART 3 – GRAPHING : Plot and Interpret Growth Curves

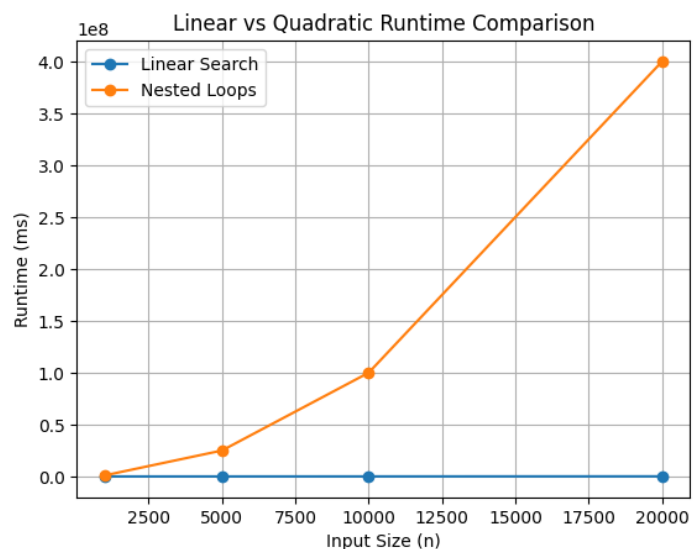
GRAPH: Excel Sheets

n	Linear	Quadratic
1000	6900	1000000
5000	28000	25000000
10000	55200	100000000
20000	110700	400000000



Data Visualization using Python

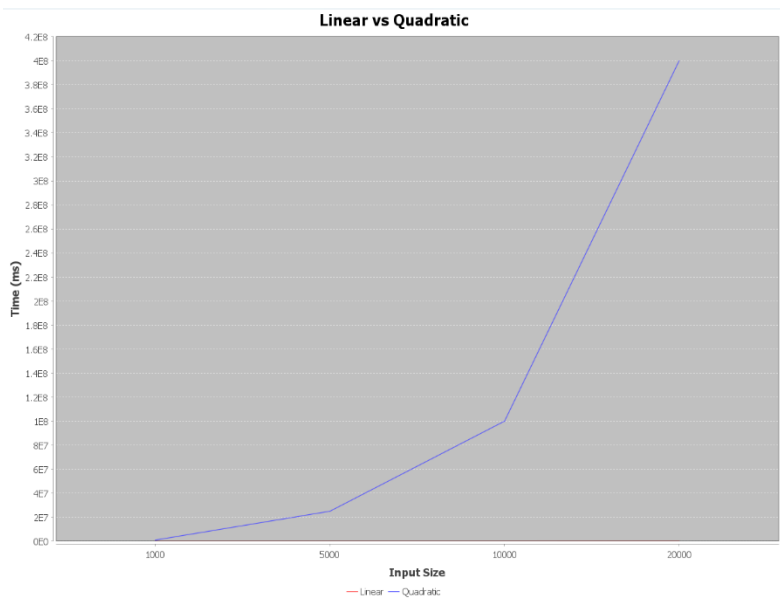
1. Goto Google Colab or any Python IDE and write these code snippet



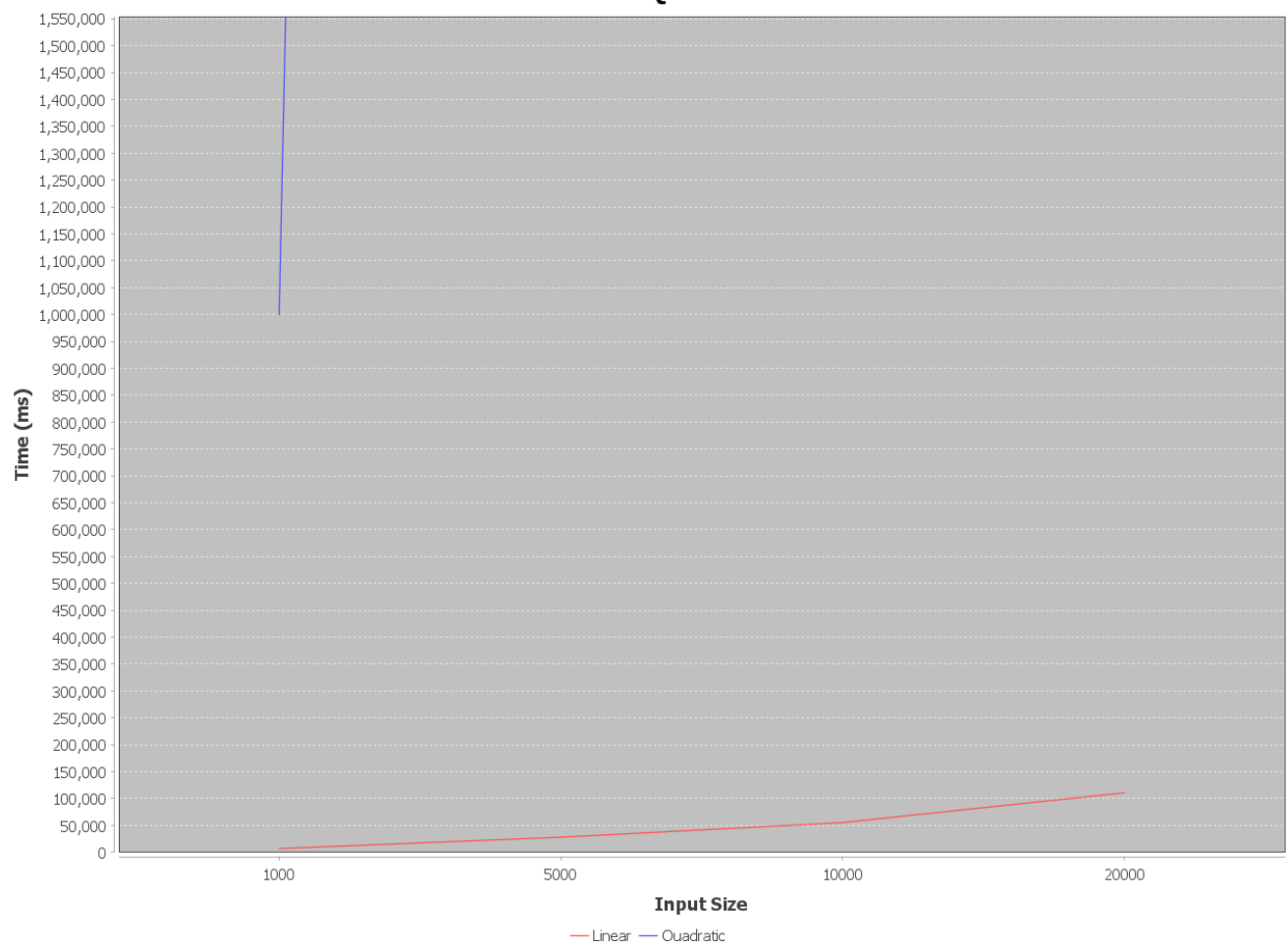


Data Visualization using Java

```
5 public class Main {
6
7     public static void main(String[] args) {
8
9         DefaultCategoryDataset dataset = new DefaultCategoryDataset();
10
11         dataset.addValue( value: 6900, rowKey: "Linear", columnKey: "1000");
12         dataset.addValue( value: 1000000, rowKey: "Quadratic", columnKey: "1000");
13
14         dataset.addValue( value: 28000, rowKey: "Linear", columnKey: "5000");
15         dataset.addValue( value: 25000000, rowKey: "Quadratic", columnKey: "5000");
16
17         dataset.addValue( value: 55200, rowKey: "Linear", columnKey: "10000");
18         dataset.addValue( value: 100000000, rowKey: "Quadratic", columnKey: "10000");
19
20         dataset.addValue( value: 110700, rowKey: "Linear", columnKey: "20000");
21         dataset.addValue( value: 400000000, rowKey: "Quadratic", columnKey: "20000");
22         // repeat for all n values
23
24         JFreeChart chart = ChartFactory.createLineChart(
25             title: "Linear vs Quadratic",
26             categoryAxisLabel: "Input Size",
27             valueAxisLabel: "Time (ms)",
28             dataset,
29             PlotOrientation.VERTICAL,
30             legend: true, tooltips: true, urls: false);
31
32         ChartFrame frame = new ChartFrame( title: "Algorithm Comparison", chart);
33         frame.pack();
34         frame.setVisible(true);
35     }
36 }
```



Linear vs Quadratic





Insights

Explain which grows faster and why.

- Based on the runtime comparison graphs, we can clearly see the difference between linear search and nested loops. Linear search grows steadily and predictably, showing $O(n)$ behavior. For example, when the data size doubles from 10,000 to 20,000 items, the runtime also doubles from 55,200 to 110,700 nanoseconds. In contrast, nested loops grow much faster, following $O(n^2)$ behavior. Doubling the data from 10,000 to 20,000 items makes the runtime four times longer, jumping from 100,000,000 to 400,000,000 nanoseconds. This creates a huge speed gap: at 20,000 items, nested loops take about 3,600 times longer than linear search, and even at just 1,000 items, they already require 145 times more time. The graphs highlight that linear search remains efficient and manageable even with large datasets, while nested loops quickly become impractical and unusable. For applications handling thousands or millions of records, choosing an $O(n)$ algorithm ensures smooth performance, whereas relying on an $O(n^2)$ algorithm risks freezing or crashing. In short, both approaches may work with small data, but only linear search stays practical as the dataset grows.

Write your short analysis (5–7 sentences).

- The results show a clear difference between linear and quadratic complexity. Linear search scales proportionally with input size and stays reasonable across all tests, while the nested loops algorithm grows much faster and becomes impractical as data increases. The $3,600\times$ gap at $n=20,000$ highlights why complexity analysis matters: both finish quickly on small inputs, but only the $O(n)$ approach remains viable as datasets grow. This is why developers should consider algorithmic efficiency early in design, before performance issues appear in production.



PART 4 – CLASSIFICATION

Algorithm	Observed Behavior	Classification O(?)	Explanation
Linear Search	Time increases proportionally with input size. Doubling n doubles the runtime.	$O(n)$	The algorithm performs a single pass through the data, checking each element once. The number of operations scales linearly with input size, resulting in predictable, manageable growth.
Nested Loops	Time increases quadratically. Doubling n quadruples the runtime. The growth curve is steep and exponential.	$O(n^2)$	The algorithm contains two nested loops, where each element is compared with every other element. This results in $n \times n$ operations, causing runtime to grow exponentially as input size increases.

PART 5 – KEY TAKEAWAYS (25 points) --- Task 5

Understanding algorithms is essential for anyone looking to excel in computer science and programming. They not only enhance problem-solving skills but also improve the efficiency and effectiveness of software solutions. As technology continues to evolve, the role of algorithms will only become more significant, making it imperative for learners and professionals alike to grasp their concepts and applications.

Imagine you are tasked with explaining algorithmic efficiency to a non-technical audience. Develop an infographic or a presentation that illustrates the concept of algorithmic efficiency using visual aids. Your submission should include a creative visualization and a clear explanation of how algorithmic efficiency impacts the performance of algorithms in real-world scenarios.

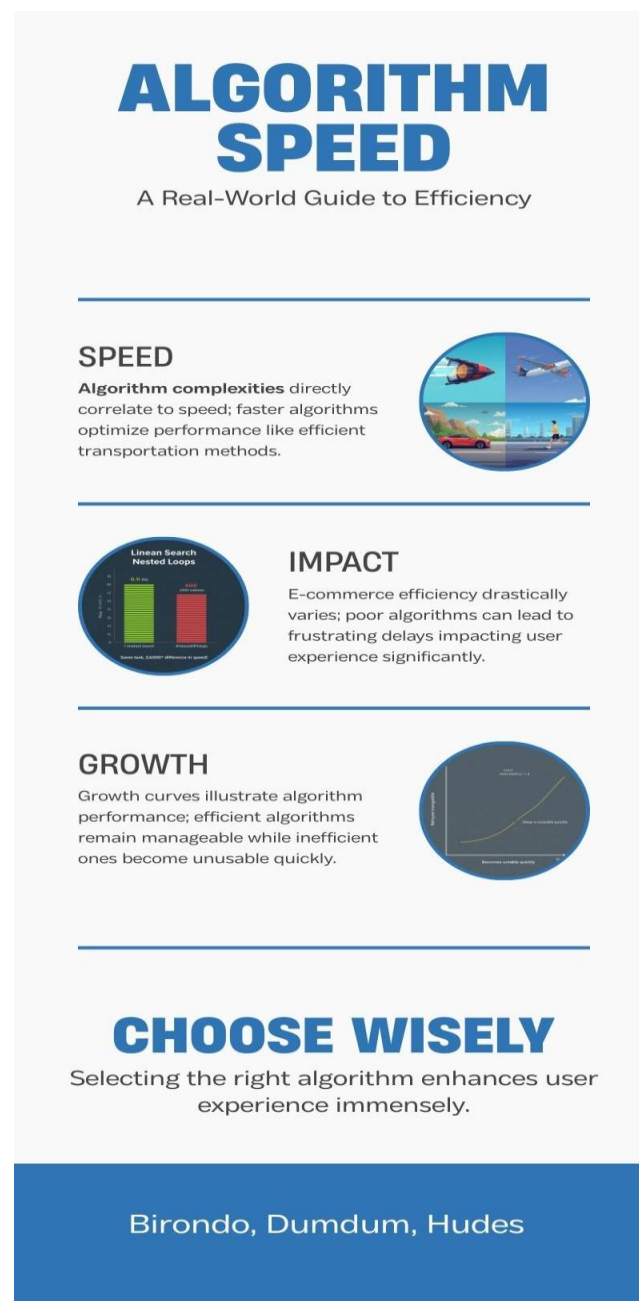




Key Take aways:

Algorithmic efficiency has a huge impact on whether software runs smoothly or struggles under real-world use. Choosing an $O(n)$ algorithm means performance scales predictably, while an $O(n^2)$ algorithm quickly becomes unusable as data grows. Everyday examples show this clearly: online shopping searches, social media feeds, banking transactions, GPS route finding, and video recommendations all depend on fast algorithms to keep users satisfied. Slow algorithms, on the other hand, cause delays, crashes, and massive costs for servers and electricity. The scaling problem is the biggest issue, what seems “fast enough” with small data can completely fail when millions of records are involved. Our lab results proved this: at 20,000 items, nested loops took 3,600 times longer than linear search, showing how quickly inefficiency becomes a disaster. In business terms, efficient algorithms save money, keep customers happy, and give companies a competitive edge, while poor choices lead to lost sales and higher costs. In short, algorithm efficiency isn't just about speed—it decides whether an app can handle real-world demands at all.

INFOGRAPHIC





7. Required Output (PDF)

Submit the following in **PDF**:

- ✓ **Lab worksheet:**
 - Operation count solution
 - Runtime table
 - Plotted graph
 - Short written analysis
 - Big-O classification
 - Key takeaways
- ✓ **Source code:** Attach as a ZIP or GitHub link

✓ **Filename format:** CS124_Lab1_LastName_FirstName.pdf

8. Assessment Rubric (100 pts)

Criteria	Excellent (15)	Proficient (10)	Developing (5)	Beginning (1)
Accuracy of Operation Counting	All operation counts correct and well explained	Minor errors but clear reasoning	Several errors; reasoning unclear	Incorrect or missing
Runtime Measurement Quality	Precise measurements with valid methodology	Mostly correct; minor inconsistency	Unclear measurement process	Missing or invalid
Graph & Interpretation	Graph is accurate, clean, and analysis insightful	Graph complete; analysis acceptable	Graph incomplete; weak analysis	Missing graph or interpretation
Classification & Justification	Correct classification with strong justification	Correct classification with basic justification	Correct but poorly justified	Incorrect or missing
Documentation & Code Quality	Clean, organized, error-free	Minor issues but readable	Messy or incomplete	Not submitted

6. Academic Integrity

- Do not share code.
- You may discuss logic, but implementation must be your own.
- Proper citation must be applied if referencing external material like ChatGPT, Github, StackOverflow, etc.

10. References

- Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
- Weiss, M. (2014). *Data Structures and Algorithm Analysis*. Pearson.

